Math for AI, at a glance

Every idea from the calculus series on one page, with its formula, the idea in a sentence, where it shows up in a neural network, and its hidden meaning, which is what it says about how a network learns.

Govinda Vurjana · https://govinda-vurjana.github.io/me/math/cheat-sheet/ · Oct 2026

THE CORE IDEA

Derivative

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

IDEA The slope at one point: how much the output moves per unit of input. Near x, a small nudge gives $\Delta f \approx f'(x)\,\Delta x$.

IN AI $\partial L / \partial w$ says whether nudging a weight raises or lowers the loss, and how fast. It is the one question training asks of every weight.

HIDDEN MEANING A derivative is blame: how much of the error is this weight's doing. It is only true nearby, which is why networks learn in small steps.

A training step

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

IDEA Move each weight a small step against its slope. The learning rate η sets the size of the step.

IN AI Every neural network trains by repeating this: a forward pass for the loss, a backward pass for the gradient, then a small step downhill.

HIDDEN MEANING Learning is not solving for the answer. It is many small corrections, each trusting a straight-line picture of the loss for one short step.

RULES

Power rule

$$\frac{d}{dx}\, x^n = n\, x^{n-1}$$

IDEA Works for any power, so rewrite first: $1/x^2 = x^{-2}$ and $\sqrt{x} = x^{1/2}$.

IN AI The squared error $(\hat{y} - y)^2$ and weight penalties like λw^2 are powers.

HIDDEN MEANING The slope of the squared error is $2(\hat{y} - y)$, proportional to the error itself: big mistakes get big corrections, nearly right answers barely move.

Sums and multiples

$$\big(af + bg\big)' = af' + bg'$$

IDEA Rates add, and scaling a function scales its slope.

IN AI A batch's loss is the average of each example's loss, so the batch gradient is the average of their gradients.

HIDDEN MEANING Every example votes on how each weight should move, and the gradient is the tally. A random mini-batch is a fair sample of the vote.

Product rule

$$(fg)' = f'g + fg'$$

IDEA Both factors change at once, and each one's change is scaled by the size of the other. It is not $f'g'$.

IN AI Gates multiplying signals in an LSTM; attention weights multiplying value vectors.

HIDDEN MEANING Multiplying is gating. A factor near zero blocks the gradient to whatever it multiplies, so the network can choose what not to learn from.

Quotient rule

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$$

IDEA Bottom times derivative of top, minus top times derivative of bottom, over bottom squared.

IN AI The sigmoid and the softmax are both quotients.

HIDDEN MEANING Dividing by a total ties the outputs together: raising one probability must lower the others, so every score's gradient touches all of them.

Chain rule

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

IDEA For a function inside a function, multiply the rates along the chain: differentiate the outside, keep the inside, times the derivative of the inside.

IN AI A network is a chain of layers. Backpropagation is the chain rule run backwards from the loss to every weight.

HIDDEN MEANING Credit assignment. A weight's share of the blame is the product of every link between it and the loss, and one weak link silences everything before it.

FUNCTIONS INSIDE NETWORKS

Exponential

$$\frac{d}{dx}\, e^{u} = e^{u}\, \frac{du}{dx}$$

IDEA e^{x} is its own slope. With an exponent, the base stays e and the exponent's derivative comes out in front.

IN AI Softmax uses e^{z} to turn any scores into positive numbers; the sigmoid is built from e^{-z}.

HIDDEN MEANING Adding becomes multiplying: raise one score by 1 and its odds against every other option are multiplied by e. A network's scores are log-odds.

Logarithm

$$\frac{d}{dx}\, \ln u = \frac{1}{u}\, \frac{du}{dx}$$

IDEA The mirror image of e^{x}, so slopes flip to reciprocals. Logs turn products into sums.

IN AI Cross-entropy is $-\ln p$. Models add log-probabilities because multiplying many small probabilities underflows to zero.

HIDDEN MEANING Multiplying becomes adding, so evidence adds up as log-probability. $-\ln p$ is surprise: training makes a model less surprised by the truth.

Sine and cosine

$$\frac{d}{dx}\, \sin x = \cos x$$
$$\frac{d}{dx}\, \cos x = -\sin x$$

IDEA The height of a point turning on a circle changes fastest as it crosses the middle, and not at all at the top.

IN AI Positional encodings in Transformers; rotary embeddings rotate queries and keys by an angle that grows with position.

HIDDEN MEANING Position becomes an angle. With rotary embeddings, the attention between two tokens depends on how far apart they are, not on where they sit.

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$
$$\sigma'(z) = \sigma(z)\,\big(1 - \sigma(z)\big)$$

IDEA Squashes any number into the range 0 to 1. Its slope is never more than 0.25.

IN AI Probabilities and LSTM gates. Chains of slopes no bigger than 0.25 make gradients vanish in deep stacks.

HIDDEN MEANING A saturated unit is certain, and so it is deaf: far from zero its slope is almost 0, and it stops learning.

ReLU

$$\mathrm{ReLU}(z) = \max(0, z)$$

$$\text{slope} = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

IDEA Passes positive inputs through unchanged and blocks negative ones.

IN AI The default activation in deep networks: a slope of exactly 1 lets gradients pass back without shrinking.

HIDDEN MEANING A switch. Units that are off pass no gradient, so each input trains only the part of the network it switched on.

Softmax with cross-entropy

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$\frac{\partial L}{\partial z_i} = p_i - y_i$$

IDEA Exponentiate the scores and divide by their sum. With $L = -\ln p_{\text{right}}$, the gradient is simply prediction minus target (y_i is 1 for the right answer, 0 otherwise).

IN AI The last layer of every language model, choosing the next token.

HIDDEN MEANING The learning signal is belief minus truth. Confident and right gives almost no gradient; confident and wrong gives a large one. Networks learn from surprise.

MANY INPUTS, MANY OUTPUTS

Partial derivative

$$\frac{\partial}{\partial x}\left(x^2 + 3xy\right) = 2x + 3y$$

IDEA Change one input and hold every other one still.

IN AI How the loss responds to one weight while all the others stay fixed. For $L = (wx + b - y)^2$, the weight's gradient is error times input.

HIDDEN MEANING "All else equal" is an assumption, true only for tiny moves. In a real step every weight moves at once and their effects interact.

Gradient

$$\nabla f = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix}$$

IDEA All the partial derivatives in one arrow. It points straight uphill, and its length is how steep the climb is.

IN AI One entry per weight, billions in a large model. Training steps along $-\nabla L$.

HIDDEN MEANING Local and greedy: it knows which way is down from here, and nothing about where the deepest valley is.

Directional derivative

$$D_{\hat{\mathbf{u}}} f = \nabla f \cdot \hat{\mathbf{u}} = \|\nabla f\| \cos\theta$$

IDEA The slope in any direction, using a vector of length 1. Largest along the gradient, zero along a contour line.

IN AI The reason $-\nabla L$ is the steepest way down, and so the direction gradient descent takes.

HIDDEN MEANING Moving along a contour changes nothing, so a gradient step spends all of its length where it counts.

Jacobian

$$J = \begin{bmatrix} \partial u/\partial x & \partial u/\partial y \\ \partial v/\partial x & \partial v/\partial y \end{bmatrix}$$

IDEA Many outputs and many inputs: one row per output, one column per input. A small change moves the outputs by about $J\,\Delta\mathbf{x}$.

IN AI The derivative of a layer. For a linear layer $\mathbf{z} = W\mathbf{x}$ it is W. Backpropagation multiplies by one layer's Jacobian at a time.

HIDDEN MEANING One matrix decides what information survives the forward pass and what learning signal survives the backward pass.

CURVATURE

Second derivative

$$y'' = \frac{d^2y}{dx^2}$$

IDEA How fast the slope itself changes. At a flat point, $y'' > 0$ is a cup (a minimum) and $y'' < 0$ is a cap (a maximum).

IN AI The curvature of the loss, which limits how large a training step can safely be.

HIDDEN MEANING The slope says which way to go; the curvature says how far you can trust it.

Learning-rate limit

$$\eta < \frac{2}{L''}$$

IDEA Each step multiplies the distance to the minimum by $1 - \eta L''$. Above 2 the steps overshoot by more every time.

IN AI Why a learning rate that is too high makes the loss blow up. The most sharply curved direction sets the limit.

HIDDEN MEANING The sharpest direction, not the typical one, sets the safe speed for the whole network.

Hessian

$$H = \begin{bmatrix} \partial^2 f/\partial x^2 & \partial^2 f/\partial x\,\partial y \\ \partial^2 f/\partial y\,\partial x & \partial^2 f/\partial y^2 \end{bmatrix}$$

IDEA Every second partial derivative: the curvature in every direction. It is symmetric, and it tells a bowl from a saddle.

IN AI The shape of the loss landscape. In high dimensions most flat points are saddles, not minima.

HIDDEN MEANING It says what kind of place training has reached: a bowl, a saddle, a narrow ravine or a broad plain. Whether flat minima generalise better is still debated.

Newton's method

$$w \leftarrow w - \frac{L'(w)}{L''(w)}$$

IDEA Divide the slope by the curvature to jump straight to the bottom of a bowl.

IN AI Too costly for big models: 10^9 weights would need a Hessian with 10^{18} entries, so curvature is approximated instead.

HIDDEN MEANING Gradient descent knows the direction but not the distance. Much of optimiser design is about cheaply estimating the distance.

INSIDE TRAINING

Backpropagation

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a_3}\,\frac{\partial a_3}{\partial a_2}\,\frac{\partial a_2}{\partial a_1}\,\frac{\partial a_1}{\partial w}$$

IDEA Multiply the local rates from the loss back to the weight, computing the shared part of the product once.

IN AI One backward pass gives every gradient for about the cost of a forward pass. It is what loss.backward() runs.

HIDDEN MEANING The forward pass run in reverse, carrying blame instead of values. A weight's update is the signal coming in times the error coming back.

Vanishing and exploding gradients

$$0.5^{20} \approx 10^{-6}, \qquad 1.5^{20} \approx 3300$$

IDEA A long product of rates below 1 shrinks to nothing; above 1 it blows up.

IN AI Why residual connections, LSTM gates and careful initialisation exist: to keep the product near 1.

HIDDEN MEANING A statement about memory: an input from 100 steps ago can only shape learning if its influence survives 100 multiplications.

Implicit differentiation

$$\frac{d}{dx}\,y^2 = 2y\,\frac{dy}{dx}$$

IDEA When y depends on x without a formula, every y-term carries a dy/dx.

IN AI Gradients through fixed points, as in deep equilibrium models, without storing every step taken to reach them.

HIDDEN MEANING An answer defined by a condition can be trained without its history. A memory that is a fixed point can be learned at the fixed point itself.

ONE NEURON, BY HAND

Forward pass

$$x = 2,\ y = 3,\ w = 0.5,\ b = 0$$

$u = wx + b$	1
$v = u^2$	1
$L = (v - y)^2$	4

Backward pass, then one step

$\partial L/\partial v = 2(v - y)$	-4
$\partial L/\partial u = -4 \cdot 2u$	-8
$\partial L/\partial w = -8 \cdot x$	-16
$\partial L/\partial b = -8 \cdot 1$	-8

STEP One step with $\eta = 0.01$ moves w to 0.66 and b to 0.08. The loss falls from 4 to about 1.08.

EASY SLIPS

- In the chain rule the power drops by one; it doesn't vanish. $(2x + 5)^4$ gives $8(2x + 5)^3$, not $8(2x + 5)$.
- $\frac{d}{dx} e^{2x} = 2e^{2x}$. The base stays e; the 2 comes from the exponent.
- $(fg)' \neq f'g'$. Use $f'g + fg'$.
- $\partial(xy)/\partial y = x$: the other variable stays as a multiplier.
- A directional derivative needs a unit vector: divide $(1, 1)$ by $\sqrt{2}$ first.
- $\ln x$ only exists for $x > 0$; $\ln|x|$ works on both sides of zero.